

Intelligenza Artificiale

Michele Castenedoli

1 ottobre 2026

Indice

1	Introduzione all'Intelligenza Artificiale	2
1.1	Definizione e Storia	2
1.2	Tipi di Intelligenza Artificiale	2
2	Intelligent Agents	7
2.1	Definizione	7
2.2	Razionalità	9
2.3	PEAS	11
2.4	Tipi di Ambiente	12
2.5	Tipi di Problemi	13
2.6	Programma Agente	14
3	Risolvere Problemi con la Ricerca	19
3.1	Problem Solving Agents	19
3.2	Come Formulare un Problema	21
3.3	Tree-Search Algorithm	22
3.4	Ricerca Non Informata	27

Capitolo 1

Introduzione all'Intelligenza Artificiale

1.1 Definizione e Storia

Nel 1950 Alan Turing pubblica un articolo intitolato “Computing Machinery and Intelligence” in cui viene proposto un esperimento per determinare se una macchina può essere considerata dotata di intelligenza. Tale esperimento, noto come *Test di Turing*, coinvolge un interrogatore umano che comunica con due entità nascoste: una **macchina** e un **essere umano**. L'interrogatore sottopone domande a entrambe le entità e, basandosi sulle risposte fornite, deve determinare quale delle due è la macchina. Se l'interrogatore non è in grado di distinguere le due parti, la macchina è considerata intelligente.

In seguito l'attenzione si è spostata sulla ricerca di metodi utili per risolvere problemi che richiedono l'intelligenza umana mediante l'utilizzo di algoritmi e modelli matematici fino ad arrivare all'introduzione di reti neurali e deep learning.

Intelligenza Artificiale

L'**Intelligenza Artificiale** è la disciplina che si occupa di studiare come **simulare** l'intelligenza umana in scenari complessi.

1.2 Tipi di Intelligenza Artificiale

1.2.1 Autonomous Agents

Gli **Autonomous Agents** sono sistemi che percepiscono l'ambiente e agiscono autonomamente per raggiungere determinati obiettivi.

1.2.2 Data Analysis

Consiste nell'utilizzo di algoritmi per analizzare grandi quantità di dati ed estrarre informazioni utili e possibili correlazioni complesse.

1.2.3 Machine Learning

È lo sviluppo di algoritmi che permettono a modelli di apprendere da *dati di esempio* e migliorare le loro prestazioni nel tempo senza essere esplicitamente programmati: a differenza della programmazione manuale e tradizionale, dove lo sviluppatore scrive una serie di istruzioni codificate in un linguaggio di programmazione, si fornisce soltanto un algoritmo di base e una grande quantità di dati da cui apprendere. Il sistema analizza tali dati per trovare possibili pattern, correlazioni e regole statistiche necessarie per svolgere il compito.

Il Machine Learning è suddiviso in tre categorie principali:

- **Unsupervised Learning:** il modello viene addestrato nel corso del tempo su un insieme di dati non etichettati (senza alcun “tag”, categoria, o classificazione assegnati a priori da un essere umano). Il target è scoprire diverse strutture nascoste o pattern nei dati senza l'utilizzo di risposte corrette predefinite.
- **Supervised Learning:** il modello viene addestrato su un insieme di dati etichettati, dove ogni esempio di input è associato a una risposta corretta. L'obiettivo è che il modello impari a mappare gli input alle risposte corrette.
- **Reinforcement Learning:** il modello impara attraverso continue interazioni con l'ambiente, ricevendo ricompense o penalità a seconda delle azioni intraprese. L'obiettivo è massimizzare la ricompensa totale nel tempo. Pertanto il sistema valuta le conseguenze delle scelte a lungo termine anziché cercare un vantaggio immediato.

Esempio: il sistema che simula un giocatore di scacchi potrebbe sacrificare la regina se prevede che in due mosse sarà in grado di dare scacco matto, raggiungendo così la massima ricompensa disponibile.

1.2.4 Time Series Analysis

La Time Series Analysis è un'area del Machine Learning che si concentra sull'analisi di dati raccolti nel tempo. Le time series sono sequenze di dati misurati a intervalli regolari (temperatura giornaliera, prezzi delle azioni o

dati di vendita mensili). In questo caso, l'obiettivo è identificare pattern, tendenze, stagionalità¹ per fare previsioni future.

Gli approcci comuni per la Time Series Analysis sono:

- **Riconoscimento di anomalie e cause:** processo di identificazione di dati o eventi che si discostano significativamente dal comportamento normale o atteso. Queste anomalie possono fornire indicatori di problemi, errori o situazioni non previste che richiedono attenzione.
- **Generative Transformers:** sono una classe di modelli che permettono di predire il prossimo elemento di una sequenza di dati partendo dagli elementi precedenti (token di un testo, pixel di un'immagine). Si introduce il concetto di **attenzione**: meccanismo usato per pesare l'importanza delle diverse parti della sequenza in input durante la generazione di output.

Esempio: Se il modello di apprendimento sta elaborando la sequenza di token relativa alla frase "ho mangiato una pesca", il meccanismo di attenzione assegnerà un peso (punteggio) molto elevato alle parole "mangiato" e "una" per aiutare il sistema a capire che ci si sta riferendo al frutto e non all'hobby di pescare.

1.2.5 Intelligent Agents

Un Intelligent Agent è un sistema che percepisce l'ambiente circostante attraverso sensori e agisce sull'ambiente per raggiungere un obiettivo specifico e svolgere un determinato compito. Gli elementi cardine di un Intelligent Agent includono:

- **Performance Measure:** misurazione del successo (efficacia) dell'agente nel raggiungere i suoi obiettivi.
- **Rationality:** l'agente deve agire in modo da massimizzare la sua performance measure attesa, dati la sequenza percettiva e la conoscenza a priori di cui dispone.

Differenza – Intelligent Agents vs Autonomous Agents

I due concetti si sovrappongono in gran parte, ma pongono l'accento su aspetti diversi. L'**intelligenza** riguarda la *qualità delle decisioni*: un agente è intelligente (razionale) se sceglie le azioni che massimizzano la performance measure attesa. L'**autonomia** riguarda invece *quanto*

¹ Stagionalità: la presenza di fluttuazioni, variazioni o pattern prevedibili che si ripetono a intervalli regolari di tempo.

l'agente dipende dal progettista: un agente è autonomo nella misura in cui il suo comportamento si basa sulla propria esperienza (percezioni e apprendimento) anziché solo sulla conoscenza inserita a priori, e opera senza un intervento umano continuo. Le due proprietà sono quindi indipendenti: un termostato con regole fisse è un agente ma non è autonomo, mentre un'auto a guida autonoma che impara e si adatta a situazioni nuove è sia intelligente sia autonoma.

1.2.6 Markov Decision Process (MDP)

Un MDP è un modello matematico utilizzato per rappresentare problemi di decisione sequenziali in ambienti stocastici. Gli elementi principali sono:

- **States:** insieme S degli stati; uno stato $s \in S$ rappresenta l'ambiente in un dato momento
- **Actions:** insieme A delle azioni che l'agente può intraprendere
- **Transition model:** effetto che le azioni hanno sull'ambiente. Poiché gli effetti potrebbero essere incerti, il modello assegna una probabilità a ogni possibile stato successivo

$$T(s, a, s') = P(s' \mid s, a)$$

dove T è la funzione di transizione ed esprime la probabilità di capitare nello stato s' sapendo che si è nello stato s e che l'azione a viene eseguita.

Approfondimento – Transition Model

Da notare che T è solo un altro modo per descrivere la probabilità condizionata di capitare in s' sapendo che si parte dallo stato s e si esegue l'azione a .

- **Reward:** valore **immediato** dell'esecuzione di un'azione

$$R : S \times A \times S \rightarrow \mathbb{R}$$

Definisce il reward che l'agente ottiene compiendo quel passo:

- Il primo S indica lo stato di partenza
- A è l'azione che è stata eseguita
- Il secondo S indica lo stato in cui si è giunti

- **Policy:** strategia che l'agente utilizza per decidere quale azione intraprendere in ogni stato con l'obiettivo di massimizzare il reward totale atteso nel tempo

$$\pi : S \rightarrow A$$

Alla base del modello c'è la **proprietà di Markov**, da cui prende il nome: la probabilità di raggiungere lo stato successivo dipende *soltanto* dallo stato corrente e dall'azione scelta, e non dalla storia degli stati e delle azioni precedenti:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0) = P(s_{t+1} \mid s_t, a_t)$$

In altre parole, lo stato corrente riassume tutta l'informazione rilevante per decidere come agire.

Capitolo 2

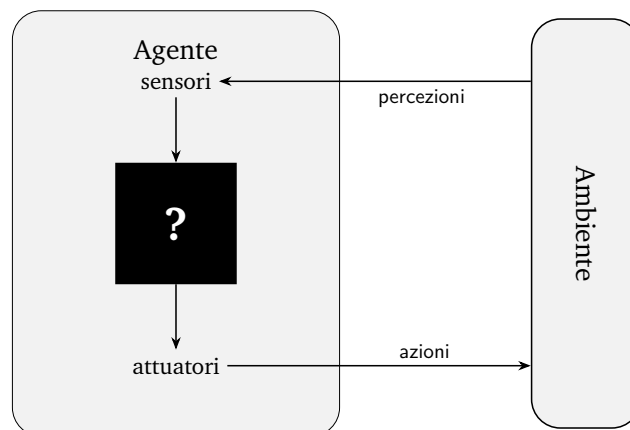
Intelligent Agents

2.1 Definizione

Intelligent Agent

Un **agente** è qualsiasi cosa possa essere vista come un sistema che percepisce il suo ambiente attraverso dei **sensori** e agisce su di esso tramite **attuatori**.

Esempio: L'agente umano possiede sensori come occhi e orecchie, mentre può usare come attuatori mani, gambe, tratto vocale. Un agente robotico potrebbe invece avere telecamere e telemetri a infrarossi come sensori e diversi motori come attuatori.



Si usa il termine **percezione** per indicare i dati che i sensori raccolgono. La **Sequenza Percettiva** di un agente è la storia di tutto ciò che l'agente

ha raccolto in tutta la sua esistenza. In generale, la scelta dell'azione di un agente in un qualsiasi istante può dipendere dalla conoscenza integrata in esso e dall'intera sequenza percettiva osservata fino a quel momento, ma non da qualcosa che l'agente non abbia percepito.

Specificando l'azione che un agente può scegliere in qualunque istante, si definisce l'agente in modo completo. In termini matematici, il comportamento di un agente è descritto dalla funzione agente, che descrive la corrispondenza tra una qualsiasi sequenza percettiva e una specifica azione.

La funzione agente assume la seguente forma:

$$f : \mathcal{P}^* \rightarrow \mathcal{A}$$

In particolare:

- $\mathcal{P}$ indica l'insieme delle singole percezioni
- $*$ (*chiusura di Kleene*) è l'operatore che, applicato a un insieme, restituisce l'insieme di *tutte* le sequenze finite (compresa quella vuota) formate dai suoi elementi
- $\mathcal{P}^*$ è quindi l'insieme di tutte le possibili sequenze percettive: ogni suo elemento rappresenta una possibile cronologia delle percezioni registrate fino a un certo momento
- $\mathcal{A}$ indica l'insieme delle possibili azioni

Internamente, la funzione agente è implementata da un **programma agente**.

Differenza – Programma Agente vs Funzione Agente

Mentre la funzione agente è una descrizione matematica astratta, il programma agente è una sua implementazione fisica concreta, in esecuzione all'interno del sistema fisico.

Esempio: Un mondo dell'aspirapolvere con due sole posizioni. Ogni posizione può essere pulita o sporca, l'agente può andare a sinistra o a destra e può pulire il riquadro in cui si trova.

Sequenza Percettiva	Azione
[A, Pulito]	Destra
[A, Sporco]	Aspira
[B, Pulito]	Sinistra
[B, Sporco]	Aspira
[A, Pulito], [A, Pulito]	Destra
[A, Pulito], [A, Sporco]	Aspira
$\vdots$	$\vdots$

Se un agente ha $|\mathcal{P}|$ possibili percezioni, dopo T passi temporali, la funzione agente ha $\sum_{t=1}^T |\mathcal{P}|^t$ voci. Se lo storico di percezioni è irrilevante, cioè se l'azione dipende soltanto dalla percezione corrente, l'agente viene detto **Simple Reflex Agent**.

L'obiettivo è progettare piccoli programmi agente per rappresentare enormi funzioni agente.

Un possibile programma agente

```
function AGENTE-REATTIVO-ASPIRAPOLVERE([posizione, stato]) returns azione
  if stato = Sporco then return Aspira
  else if posizione = A then return Destra
  else if posizione = B then return Sinistra
```

Questo programma è adeguato se la performance measure prende in considerazione solo il numero di stanze pulite. Se subentrassero altre metriche che portano a una definizione della performance measure diversa (stato della batteria, percentuale di ricarica, minimizzare il numero di spostamenti), tale programma non sarebbe in grado di esprimere la cosa migliore da fare. Da notare che l'agente che implementa tale programma è un **Simple Reflex Agent**: l'azione dipende soltanto dalla percezione corrente.

2.2 Razionalità

2.2.1 Comportarsi Correttamente: il concetto di razionalità

Definizione

Un **agente razionale** è un agente che fa la cosa giusta.

Esistono diverse accezioni dell'espressione "cosa giusta", ma l'IA rimane fedele alla nozione di **conseguenzialismo**: si valuta il comportamento di un agente considerando le conseguenze. Quando un agente viene inserito in un ambiente, genera una sequenza di azioni in base alle percezioni che riceve. Questa sequenza di azioni porta l'ambiente ad attraversare una sequenza di stati: se tale sequenza è desiderabile, significa che l'agente si è comportato bene. Questa nozione di desiderabilità è catturata da una performance measure che valuta la sequenza di stati dell'ambiente.

In un dato momento, ciò che è razionale dipende da quattro fattori:

- la performance measure che definisce il criterio del successo
- la conoscenza pregressa dell'ambiente da parte dell'agente

- le azioni che l'agente può eseguire
- la sequenza percettiva dell'agente fino allo stato corrente

Tutto ciò porta a una definizione molto formale di **agente razionale**:

Definizione

Per ogni possibile sequenza di percezioni, un agente razionale dovrebbe scegliere un'azione che massimizzi il valore atteso della sua performance measure, date le informazioni fornite dalla sequenza percettiva e da ogni ulteriore conoscenza dell'agente.

La razionalità allora non è altro che un problema di ottimizzazione sulla performance measure: un agente razionale sceglie l'azione che massimizza il valore atteso della performance measure, data la sequenza percettiva accumulata fino a quel momento.

Notare come essere razionale non vuol dire essere onnisciente.¹ Infatti le percezioni potrebbero non fornire tutte le informazioni rilevanti in un dato istante. Pertanto, i risultati potrebbero essere diversi da quelli previsti.

Differenza – Razionalità vs Perfezione

La razionalità massimizza la performance **attesa**, mentre la perfezione massimizzerebbe la performance **effettiva**. Un agente razionale non può essere giudicato per esiti che non poteva prevedere: se attraverso la strada dopo aver controllato che sia libera e vengo colpito da un oggetto caduto dall'alto, la mia scelta è stata comunque razionale.

2.2.2 Esercizio

L'esercizio propone di analizzare un ambiente composto da tre stanze (A, B, C) e due robot (r_1, r_2). Il robot r_1 può pattugliare le stanze A e B partendo da A, mentre r_2 può pattugliare B e C partendo da C. Assumendo un tempo di spostamento nullo, l'obiettivo è definire un comportamento razionale capace di minimizzare l'inattività media (*average idleness*), definita come la frazione del tempo in cui una stanza non è visitata da alcun robot.

Per affrontare il problema, possiamo analizzare tre diverse strategie:

- **Prima strategia (Vettore di inattività $[0, 1, 0]$):** I robot non cambiano mai la loro posizione iniziale. Il robot r_1 rimane in A e r_2 in C. Di

¹Possedere conoscenza illimitata su qualsiasi cosa.

conseguenza, le stanze A e C hanno inattività nulla, ma la stanza B non viene mai coperta, generando un'inattività pari a 1.

- **Seconda strategia (Vettore di inattività $[1/2, 1/2, 0]$):** Si decide di muovere soltanto il primo robot (r_1), che si sposterà continuamente da A a B e viceversa, mentre r_2 resta fermo in C. In questo scenario, le stanze A e B rimangono scoperte per metà del tempo.
- **Terza strategia (Vettore di inattività $[1/4, 1/2, 1/4]$):** Questa strategia consente di far giocare un ruolo attivo anche al secondo robot (r_2), coordinando il movimento di entrambi per coprire l'intera area: ogni robot trascorre $3/4$ del tempo nella propria stanza esterna e $1/4$ in B, e i due non si trovano mai in B contemporaneamente. La stanza B resta quindi coperta per metà del tempo.

Si noti che in tutte e tre le strategie la somma delle inattività vale 1, quindi l'**inattività media è sempre** $1/3$: rispetto all'obiettivo formale le tre strategie sono equivalenti. Tuttavia la **terza strategia è la migliore**: nella pratica, per coprire il più possibile tutte le diverse stanze in modo omogeneo, è fondamentale introdurre il concetto di *equità*. Per garantire questa equità, limitarsi a valutare la media non è sufficiente: risulta necessario introdurre il calcolo della **varianza** o della **deviazione standard**, in modo da penalizzare gli squilibri estremi tra le stanze.

2.3 PEAS

Per progettare un agente intelligente bisogna definire l'ambiente in cui opera:

- **Performance Measures:** metodo di valutazione del successo dell'agente
- **Environment:** il contesto in cui l'agente opera
- **Actuators:** i mezzi attraverso cui l'agente agisce sull'ambiente
- **Sensors:** i mezzi attraverso cui l'agente percepisce l'ambiente

Esempio: Prendiamo in considerazione un taxi automatico. Il PEAS potrebbe essere:

Performance measures	Soddisfazione del cliente, sicurezza, efficienza del carburante (SOLO E85), rispetto per le leggi stradali
Environment	Traffico stradale, condizioni meteorologiche, segnali stradali, pedoni e altri veicoli
Actuators	Volante, acceleratore, freni, indicatori di direzione
Sensors	Telecamere, GPS, sensori di velocità, radar

2.4 Tipi di Ambiente

Gli ambienti possono essere classificati secondo diverse caratteristiche:

- **Osservabile** (vs *parzialmente osservabile*): se l'agente può percepire completamente lo stato dell'ambiente in ogni momento
- **Deterministico** (vs *stocastico*): se lo stato successivo dell'ambiente è determinato completamente dallo stato corrente e dall'azione eseguita dall'agente
- **Episodico** (vs *sequenziale*): se l'esperienza dell'agente è divisa in episodi indipendenti, cioè l'azione in un episodio non influisce sugli episodi successivi
- **Statico** (vs *dinamico*): se l'ambiente non cambia mentre l'agente sta prendendo una decisione
- **Discreto** (vs *continuo*): se stati, percezioni, azioni e tempo assumono un numero finito di valori distinti
- **Singolo Agente** (vs *multi-agente*): se l'agente opera da solo nell'ambiente senza la presenza di altri agenti
- **Noto** (vs *ignoto*): se l'agente (o il progettista) conosce le regole dell'ambiente, cioè gli esiti delle azioni

Approfondimento – Scenario Singolo Agente in contesti Multi-Agente

Un ambiente multi-agente può essere considerato singolo agente in quelle circostanze in cui si prevede la progettazione di un singolo

agente mentre gli altri agenti non vengono modellati (che quindi vengono considerati elementi dell'ambiente).

Esempio: Tipi di ambienti in diversi contesti

	Crossword	Robo-selector	Poker	Taxi
Observable	Yes	Partly	Partly	Partly
Deterministic	Yes	No	No	No
Episodic	No	Yes	No	No
Static	Yes	No	Yes	No
Discrete	Yes	No	Yes	No
Single-agent	Yes	Yes	No	No

Durante il corso ci soffermeremo sugli ambienti *osservabili* e *deterministici* per descrivere problemi sequenziali: un'azione svolta ha un impatto sul futuro.

2.5 Tipi di Problemi

- **Deterministico, completamente osservabile $\Rightarrow$ problema a stato singolo (Single-State Problem).**
L'agente sa esattamente in quale stato si troverà; la soluzione è una sequenza di azioni.
- **Non osservabile $\Rightarrow$ problema conformante.**
L'agente potrebbe non avere idea di dove si trovi: l'agente non dispone di sensori. La soluzione (se esiste) è una sequenza di azioni.
- **Non deterministico e/o parzialmente osservabile $\Rightarrow$ problema di contingenza.**
Le percezioni forniscono costantemente nuove informazioni sullo stato corrente; la soluzione è un piano contingente (basato su condizioni del tipo if-then) o una *policy* (mapping stato-azione): molto spesso si alternano le fasi di ricerca e di esecuzione.
- **Spazio degli stati sconosciuto $\Rightarrow$ problema di esplorazione (*online*).**
L'agente deve scoprire gli stati e le transizioni interagendo direttamente con l'ambiente per capire come funziona il problema.

2.6 Programma Agente

Poiché l'agente è composto da un'architettura interna e un **programma agente**, è utile definire le caratteristiche generali di un programma agente.

In generale, lo scheletro di un programma è composto da:

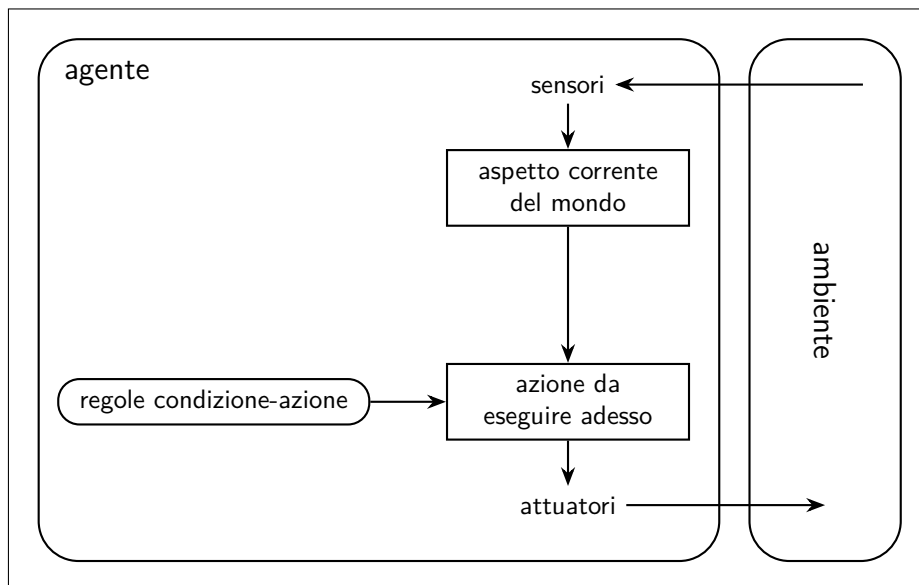
- Input: Percezione **corrente**. In tutti quei contesti in cui l'agente necessita di una sequenza di percezioni, ogni percezione dovrà essere salvata per riassemblare l'intera sequenza
- Output: **Prossima** azione

In questo schema è possibile seguire alcune architetture che consentono la modellazione di diversi agenti. Architetture diverse hanno diversi scopi. Di seguito l'elenco dei quattro tipi base di quasi tutti i sistemi intelligenti:

- Simple Reflex Agents
- Reflex Agents with State
- Goal-Based Agents
- Utility-Based Agents

2.6.1 Simple Reflex Agents

L'architettura più semplice è il **Simple Reflex Agent**. Questi agenti scelgono le azioni sulla base della percezione corrente, ignorando tutta la storia percettiva precedente. Pertanto, il principio seguito è quello del tipo **regola condizione-azione**.



Simple Reflex Agents

```
function AGENTE-REATTIVO-SEMPLICE(percezione) returns azione
  persistent: regole          // insieme di regole condizione-azione

  stato ← INTERPRETA-INPUT(percezione)
  regola ← REGOLA-CORRISPONDENTE(stato, regole)
  azione ← regola.AZIONE
  return azione
```

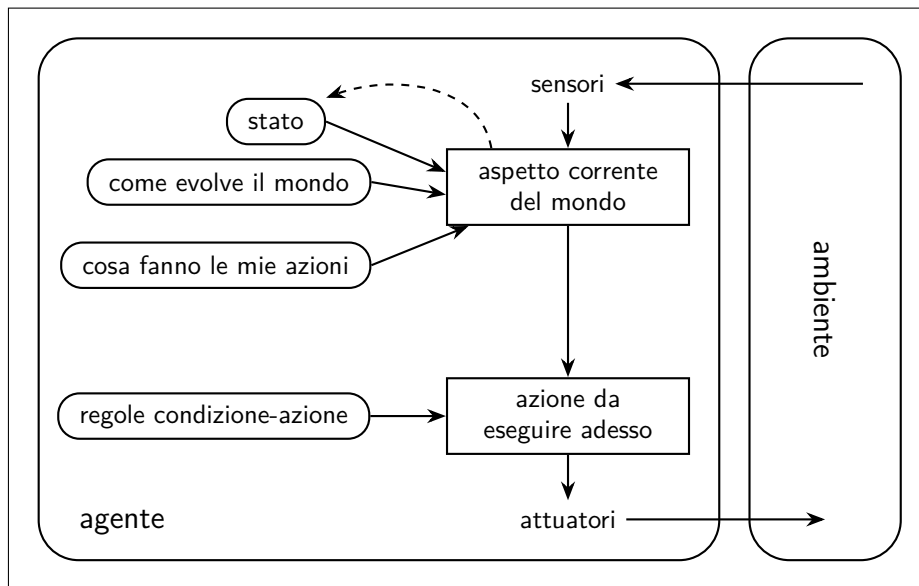
I Simple Reflex Agents hanno il pregio della semplicità, ma un'intelligenza molto limitata: funzionano correttamente **solo se l'ambiente è completamente osservabile**, cioè se la decisione giusta può essere presa sulla base della sola percezione corrente. Anche una minima parte di non osservabilità può causare seri problemi.

Esempio: se l'aspirapolvere non avesse il sensore di posizione e percepisce soltanto [Pulito] o [Sporco], davanti a una stanza pulita dovrebbe scegliere sempre la stessa azione, ad esempio *Sinistra*. Se si trova in A, quell'azione fallisce sempre: l'agente entra in un **ciclo infinito**.

2.6.2 Reflex Agents with State

Il modo più efficace di gestire l'osservabilità parziale, per un agente, è tener traccia della parte del mondo che non può vedere nell'istante corrente. Questo significa che l'agente deve mantenere una sorta di **stato interno** che dipende dalla storia delle percezioni e che quindi riflette almeno una parte degli aspetti non osservabili dello stato corrente. In particolare, aggiornare lo stato interno richiede due componenti fondamentali: gli effetti delle azioni dell'agente e le modalità di evoluzione del mondo indipendentemente dall'agente. Tale conoscenza viene implementata tramite un **transition model**. Inoltre, sono necessari **modelli sensoriali** che descrivono come lo stato del mondo si rifletta nelle percezioni dell'agente.

Un agente che utilizza tali modelli prende il nome di **Agente Basato su Modello**.



Reflex Agents with State

```
function AGENTE-REATTIVO-BASATO-SU-MODELLO(percezione) returns azione
persistent:
    stato                // concezione corrente dello stato del mondo
    modello_transizione // stato successivo in funzione di stato e azione
    modello_sensoriale  // come lo stato si riflette nelle percezioni
    regole              // insieme di regole condizione-azione
    azione              // l'azione più recente, inizialmente nessuna

    stato ← AGGIORNA-STATO(stato, azione, percezione,
                           modello_transizione, modello_sensoriale)
    regola ← REGOLA-CORRISPONDENTE(stato, regole)
    azione ← regola.AZIONE
    return azione
```

2.6.3 Goal-Based Agents

La conoscenza dello stato corrente dell'ambiente non è sempre sufficiente per decidere quale azione intraprendere. Spesso è necessario introdurre informazioni relative all'**obiettivo** (*goal*), ovvero la descrizione delle situazioni desiderabili da raggiungere. Un *Goal-Based Agent* combina queste informazioni con il proprio modello interno del mondo, superando il limite delle semplici regole "condizione-azione". Non basta più, infatti, associare direttamente una percezione a un'azione prestabilita. A differenza dei *Reflex Agents*, in cui l'obiettivo è solo implicito nelle regole pre-programmate, il *Goal-Based Agent* valuta in modo esplicito le conseguenze delle proprie scelte: prima di

agire, analizza attivamente se lo svolgimento di una determinata azione lo porterà o meno al soddisfacimento del proprio scopo.

Esempio: Il Reflex Agent frena quando vede le luci di frenata, senza rendersi conto del motivo. Un Goal-Based Agent frena quando vede le luci di frenata perché quella è l'unica azione che, secondo la sua previsione, permette di raggiungere l'obiettivo di non tamponare altre macchine.

Ricerca e pianificazione sono sottocampi dell'IA dedicati a identificare le sequenze di azioni da intraprendere per raggiungere gli obiettivi.

Può sembrare che un Goal-Based Agent sia meno efficiente (invece di applicare una regola già pronta, prima di agire deve simulare le conseguenze delle possibili azioni e verificare se portano all'obiettivo), ma garantisce maggiore **flessibilità**, perché la conoscenza che guida le sue decisioni è rappresentata esplicitamente e può essere modificata.

2.6.4 Utility-Based Agents

Raggiungere semplicemente un obiettivo non basta: per un risultato ottimale è fondamentale valutare anche la qualità e l'efficienza delle azioni scelte per ottenerlo (come velocità, sicurezza o convenienza).

In questo ambito, infatti, si introduce il concetto di **Utilità**: all'interno dell'agente viene implementata una **Funzione di utilità** che internalizza la performance measure. Purché la funzione di utilità interna e la performance measure esterna concordino, un agente che sceglie le azioni per massimizzare l'utilità sarà comunque razionale in base alla performance measure esterna.

Si sottolinea che questo non è l'unico modo per essere razionali: l'esempio sull'aspirapolvere mostra la modellazione di un comportamento razionale senza introdurre obiettivi o funzioni di utilità. Anche in questo caso, comunque, i vantaggi sono principalmente in termini di flessibilità, soprattutto in contesti in cui è necessario prendere decisioni in condizioni d'incertezza.

In termini tecnici, un Utility-Based Agent razionale sceglie l'azione che massimizza l'**utilità attesa** dei risultati, ovvero l'utilità che l'agente si attende di ottenere, in media, date le probabilità e le utilità di ciascun risultato.

Esempio: Si immagina un taxi a guida autonoma che deve scegliere tra due strade:

- **STRADA A (Urbana):** è sempre libera (100%) e porta a destinazione nei tempi previsti, con utilità moderata pari a 5.
Utilità attesa: $1.0 \times 5 = 5$
- **STRADA B (Autostrada):** è libera nell'80% dei casi, offrendo la possibilità di arrivare in largo anticipo, con utilità elevata pari a 10. Nel restante 20% dei casi, invece, si trova un ingorgo totale e si arriva molto in ritardo, con utilità = -5.
Utilità attesa: $(0.8 \times 10) + (0.2 \times -5) = 8 - 1 = 7$

La strada che massimizza l'utilità attesa è pertanto la B.

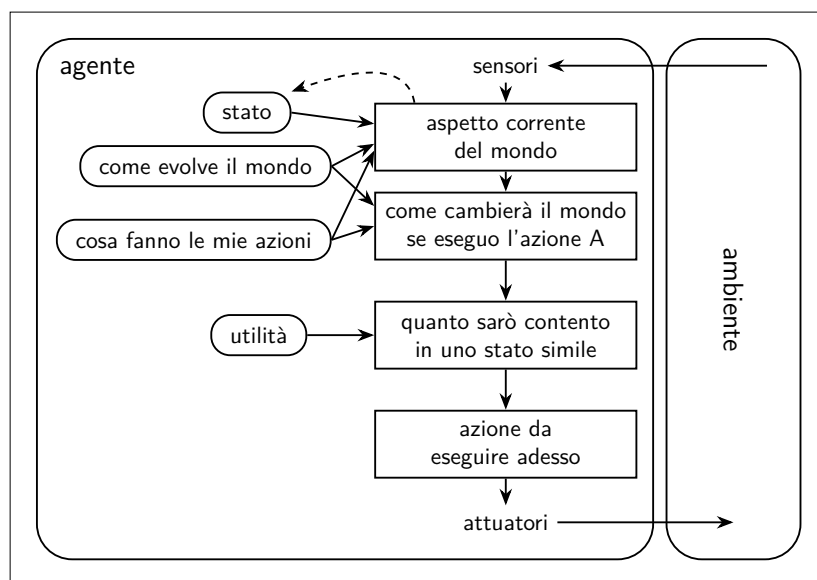
Approfondimento – Utilità Attesa

Si noti che non è altro che il calcolo del valore medio (*speranza matematica*): $E[X] = \sum x_i \cdot P(X = x_i)$

Differenza – Funzione di Utilità vs Performance Measure

Sebbene rappresentino concetti assai simili, la Performance Measure è il metro di giudizio **esterno**, stabilito dal progettista per valutare la qualità complessiva del comportamento dell'agente: l'agente non ha necessariamente accesso diretto a essa.

La funzione di utilità è invece **interna** all'agente: è l'internalizzazione della performance measure, costruita dal progettista in modo che le due siano allineate. L'agente la usa per assegnare un valore a ogni possibile stato del mondo e, in condizioni di incertezza, sceglie l'azione che massimizza l'utilità attesa. Se la funzione di utilità non rispecchia fedelmente la performance measure, l'agente può massimizzarla e comportarsi comunque in modo non razionale rispetto al criterio esterno.



Capitolo 3

Risolvere Problemi con la Ricerca

3.1 Problem Solving Agents

Quando l'azione da svolgere non è subito evidente, un agente potrebbe avere la necessità di guardare avanti, cioè considerare una sequenza di azioni (*cammino*) che porta a uno stato obiettivo. Questo tipo di agente viene chiamato **Problem Solving Agent**. I Problem Solving Agents utilizzano rappresentazioni **atomiche** degli stati, i quali vengono considerati come entità prive di una struttura interna.

I Problem Solving Agents sono in particolare una sottocategoria dei Goal-Based Agents. Per decidere quale azione intraprendere, questi agenti adottano un approccio sistematico sequenziale basato sulla formulazione di obiettivi e problemi e sulla ricerca di soluzioni. Tale processo si divide in quattro macro-fasi:

1. **Formulazione dell'obiettivo (*Goal Formulation*)**: dato lo stato corrente dell'ambiente, l'agente definisce una situazione desiderabile (l'obiettivo) da raggiungere.
2. **Formulazione del problema (*Problem Formulation*)**: l'agente decide quali stati e quali azioni prendere in considerazione per raggiungere tale obiettivo.
3. **Ricerca (*Search*)**: l'agente esplora lo spazio degli stati simulando diverse sequenze di azioni nel proprio modello interno, fino a trovare una soluzione valida (una sequenza di mosse che porta all'obiettivo).
4. **Esecuzione (*Execution*)**: l'agente esegue, una alla volta, le azioni previste dalla soluzione trovata, ignorando le percezioni.

Notare come la risoluzione del problema in questo caso è di tipo **offline**: la soluzione viene eseguita a “occhi chiusi” (come detto prima, si ignorano le percezioni).

Differenza – Online vs Offline Problem Solving

A differenza della tecnica offline, la quale può essere usata in ambienti completamente deterministici e osservabili, l'**Online Problem Solving** prevede che l'agente agisca in condizioni di parziale ignoranza, dove la conoscenza sull'ambiente non è completa. Pertanto è indispensabile che i sensori rimangano attivi per valutare l'esito delle proprie azioni.

Di seguito un'implementazione di un programma agente per un Problem Solving Agent che procede alla risoluzione di un problema mediante ricerca nello spazio degli stati.

Simple Problem-Solving Agent

```
function AGENTE-RISOLUTORE-SEMPLICE(percezione) returns azione
  persistent:
    seq          // sequenza di azioni, inizialmente vuota
    stato        // descrizione dello stato corrente del mondo
    obiettivo    // un obiettivo, inizialmente nullo
    problema     // la formulazione del problema

  stato ← AGGIORNA-STATO(stato, percezione)
  if seq è vuota then // la prima volta che si esegue
    obiettivo ← FORMULA-OBIETTIVO(stato) // che si vuole raggiungere
    problema ← FORMULA-PROBLEMA(stato, obiettivo)
    seq ← RICERCA(problema)
    if seq = failure then return null
  azione ← PRIMO(seq)
  seq ← RESTO(seq)
  return azione
```

Da notare come l'agente decida cosa fare **solamente** quando la sequenza seq è vuota: in quel momento formula obiettivo e problema e cerca una soluzione, poi la esegue ignorando completamente le percezioni. Solo una volta esaurita la sequenza torna a formulare un nuovo obiettivo.

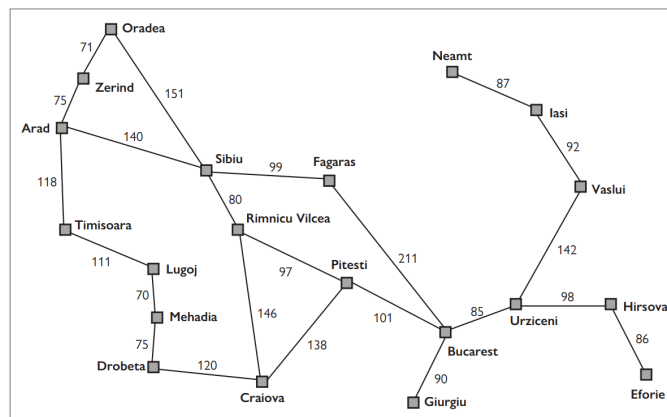
3.2 Come Formulare un Problema

3.2.1 Problema: Vacanze in Romania (con un M5 e39)

Ci troviamo in Romania e dobbiamo andare a Bucarest per poi prendere l'aereo e tornare a casa. Attualmente ci troviamo ad Arad. Vogliamo trovare la serie di città da attraversare per giungere a Bucarest. Dobbiamo individuare un corretto livello di astrazione, ma ragionevolmente si procede nel seguente modo:

- **Obiettivo:** andare a Bucarest
- **Formulazione del problema**
 1. **states:** le diverse città
 2. **actions:** guidare per attraversare le città
- **Soluzione da Individuare:** sequenza di città da attraversare per arrivare a Bucarest

Fin da subito è possibile mettere in discussione il livello di determinismo del problema. A causa di agenti esterni (incidenti, lavori) si potrebbe essere dirottati su città non scelte. Tali eventualità vengono inizialmente ignorate. In questo caso vogliamo trovare **una** soluzione: non è detto che quella trovata sia a costo minimo.



Cerchiamo di proporre una formulazione di tale quesito come Single-State Problem. Allora si individuano i seguenti elementi:

1. **Stato Iniziale:** la città di partenza.
2. **Successor Function $S(x)$:** dato uno stato x , restituisce l'insieme delle coppie azione–stato, ovvero ogni azione applicabile in x insieme allo stato in cui porta. Esempio: $S(A) = \{\langle \text{Arad} \rightarrow \text{Zerind}, \text{Zerind} \rangle, \dots\}$

3. **Goal Test** che può essere di due nature diverse:
 - (a) esplicito: la città di destinazione (Bucarest). Lo stato in cui ci si deve trovare affinché il problema possa considerarsi risolto.
 - (b) implicito: una feature che descrive più di uno stato.
4. **Path Cost**: Non sempre si desidera trovare una soluzione qualunque. Potrebbe essere utile trovare quella a costo minimo. Nel nostro esempio corrisponde a una metrica che indica il costo di un percorso (il costo espresso sugli archi). Il path cost è una funzione additiva in cui vengono sommati gli **step cost**, funzioni del tipo $\text{STEP-COST} : S \times A \times S \rightarrow \mathbb{R}^+$.
5. **Soluzione**: sequenza di azioni che porta dallo stato iniziale a uno dei goal state.

3.3 Tree-Search Algorithm

L'idea di base è fare un'esplorazione simulata e offline: essendo un Single-State Problem, le percezioni vengono ignorate. Concettualmente l'agente ragiona, individua una sequenza di azioni e procede a eseguirla. Pertanto si generano i successori degli stati usando le regole note del mondo.

Tree-Search Algorithm

```
function TREE-SEARCH(problema, strategia) returns una soluzione, o failure
  inizializza l'albero di ricerca usando lo stato iniziale del problema

  loop do
    if nessun candidato per l'espansione then return failure
    scegli uno dei nodi per l'espansione seguendo la strategia
    if il nodo contiene un goal state then return la soluzione
    else espandi il nodo e aggiungi i successori all'albero di ricerca
  end
```

3.3.1 Stati vs Nodi

Nell'albero di ricerca è importante distinguere due concetti che a prima vista possono sembrare equivalenti:

- uno **stato** è la rappresentazione di una configurazione fisica del mondo: descrive *dove* si trova l'agente (ad esempio, “sono ad Arad”);
- un **nodo** è una **struttura dati** che costituisce parte dell'albero di ricerca: oltre allo stato a cui corrisponde, memorizza *come* ci si è arrivati.

I campi di un nodo sono:

State	lo stato del mondo a cui corrisponde il nodo
Parent	il nodo da cui è stato generato
Action	l'azione applicata al padre per generare il nodo
Children	i nodi generati espandendo il nodo
Depth	il numero di passi dalla radice
Path cost $g(n)$	il costo del cammino dalla radice fino al nodo

Differenza – Stato vs Nodo

Gli stati **non** hanno padre, azioni, figli, profondità o path cost: queste informazioni dipendono dal *percorso* seguito e non dal mondo. Per questo lo **stesso stato** può comparire in **più nodi diversi** dell'albero di ricerca.

Esempio: nel problema della Romania, lo stato “sono a Sibiu” può essere raggiunto:

- direttamente da Arad: profondità 1, $g = 140$;
- passando per Arad $\rightarrow$ Zerind $\rightarrow$ Oradea $\rightarrow$ Sibiu:
profondità 3, $g = 75 + 71 + 151 = 297$.

Si ottengono quindi due nodi distinti, con padre, profondità e costo diversi, che contengono però lo stesso stato. In sintesi: lo stato indica *dove* si trova l'agente, il nodo indica anche *come* ci è arrivato.

3.3.2 Un'implementazione di più basso livello

Lo pseudocodice sopra descritto risulta essere di carattere estremamente generale, pertanto si pone a un livello di astrazione tale che risulta estremamente complesso procedere con una sua implementazione. Ecco perché generalmente si utilizza uno pseudocodice che fornisce una descrizione di più basso livello.

Tree-Search Algorithm

```
function TREE-SEARCH(problema, frontiera) returns una soluzione, o failure
  frontiera  $\leftarrow$  INSERISCI(CREA-NODO(problema.stato_iniziale))
  while not VUOTA(frontiera) do
    nodo  $\leftarrow$  POP(frontiera)
    if problema.GOAL_TEST(nodo.stato) then return nodo
    frontiera  $\leftarrow$  INSERISCI_TUTTI(ESPANDI(nodo, problema))
  end loop
  return failure
```

La frontiera non è altro che una struttura dati che contiene nodi, disposti in un certo ordine. Il modo in cui i nodi vengono inseriti nella frontiera determina il tipo di strategia che si sta utilizzando (Breadth First Search, Depth First Search, Best First Search ...).

Notiamo come la strategia non è più un parametro in input, ma è intrinseca nel modo in cui i nodi vengono inseriti nella frontiera. Prima di tutto, inseriamo nella frontiera il nodo corrispondente allo stato iniziale. Fino a quando la frontiera non è vuota si itera estraendo il primo nodo dalla frontiera (i nodi sono ordinati secondo la priorità definita dalla strategia) e si controlla se il nodo contiene il *goal state*. In caso affermativo, il nodo viene restituito, altrimenti si espande il nodo appena estratto dalla frontiera basandosi sulle regole presenti nel problema e poi si procede a inserire ciò che si è ottenuto con l'espansione all'interno della frontiera seguendo un determinato ordine. L'ordine stesso determina il modo in cui si effettua la ricerca.

Di seguito lo pseudocodice della funzione ESPANDI:

Expand Function

```
function ESPANDI(nodo, problema) returns un insieme di nodi
    successori ← INSIEME-VUOTO()
    for each (azione, risultato) in FUNZIONE-SUCCESSORI(problema, nodo.stato) do
        s ← CREA-NODO()
        s.padre ← nodo
        s.azione ← azione
        s.stato ← risultato
        s.path_cost ← nodo.path_cost +
                               STEP-COST(nodo.stato, azione, risultato)

        s.profondita ← nodo.profondita + 1
        AGGIUNGI(s, successori)
    end loop
    return successori
```

In particolare, analizziamo il suo comportamento:

1. La funzione FUNZIONE-SUCCESSORI implementa la Successor Function $S(x)$: restituisce l'insieme delle coppie (azione, risultato), cioè tutte le azioni applicabili nello stato del nodo e gli stati che ne risultano. Se un'azione non altera l'ambiente, si rimane nello stesso stato di partenza.
2. Il nuovo nodo non contiene solo lo stato di arrivo isolato, ma registra esplicitamente il nodo padre (attributo padre) e l'azione eseguita (attributo azione). Questa serie di puntatori è ciò che permet-

te di ricostruire l'intero percorso (la soluzione) una volta raggiunto l'obiettivo.

3. Il calcolo `nodo.path_cost + STEP-COST` accumula il costo effettivo del percorso dalla radice fino a quel punto esatto.
4. Raccolta dei risultati: tutti i nuovi nodi istanziati e completamente popolati vengono aggiunti all'insieme `successori`, che viene poi restituito al chiamante.

3.3.3 Search Strategies

Una strategia è definita scegliendo l'**ordine di espansione dei nodi**. Le strategie sono valutate in base alle seguenti dimensioni:

- **completezza**: se c'è una soluzione, la ricerca è in grado di trovarla.
- **complessità temporale**: in funzione del numero di nodi generati/espansi, poiché generare e valutare un nodo richiede tempo.
- **complessità spaziale**: numero massimo di nodi in memoria. Tutti i nodi che sono in memoria sono stati generati, ma potrebbe essere necessario tenere in memoria meno nodi rispetto a quelli generati.
- **ottimalità**: trova sempre la soluzione a costo minimo (quindi si desidera ottimizzare il `path_cost`, non la profondità)

Differenza – Generated vs Expanded

Qui il comportamento delle due procedure è molto diverso: la **generazione** procede a istanziare un nuovo nodo allocando lo spazio necessario e inizializzandone gli attributi, analogamente al concetto di costruttore nella OOP. L'espansione, invece, prende un nodo e le regole di un problema per procedere a generare tutti i nodi figli da poi inserire nella frontiera.

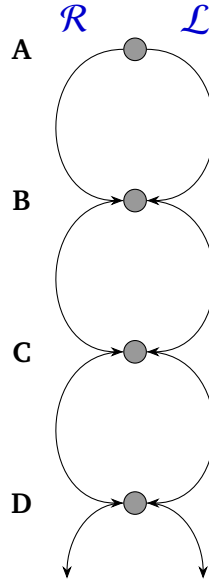
La complessità temporale e quella spaziale sono misurate in termini di:

- *b*: fattore di diramazione massimo dell'albero di ricerca. In parole meno tecniche, il massimo numero di figli di ciascun nodo.
- *d*: profondità della soluzione a costo minimo.
- *m*: profondità massima dello spazio degli stati (può essere ∞)

Notiamo come *m* è dato dallo spazio degli stati, mentre *d* dalla soluzione a costo minimo (tipicamente $d \ll m$).

3.3.4 Stati Ripetuti

Dato uno stato, possono esistere diverse azioni che consentono di raggiungere lo stesso stato successivo.



Si osservi che lo stato B viene rappresentato nella frontiera **da 2 nodi distinti**: il primo generato dall'azione $\mathcal{R}$ e il secondo dall'azione $\mathcal{L}$. Questa ridondanza fa sì che la complessità del problema cresca notevolmente, assumendo un andamento di tipo esponenziale.

Sebbene in generale si preferisca evitare l'espansione di stati ripetuti, è fondamentale prestare attenzione poiché, in certi contesti, mantenere l'esplorazione di questi percorsi multipli è cruciale per garantire l'ottimalità. Infatti, se un nodo appena espanso corrisponde al goal state e la ricerca viene interrotta immediatamente, si rischia di accettare una soluzione non ottimale.

Facendo riferimento alla figura, supponiamo che B sia il goal state e che lo step cost associato all'azione $\mathcal{R}$ sia maggiore rispetto a quello di $\mathcal{L}$. Se l'algoritmo si fermasse all'istante subito dopo aver raggiunto B tramite $\mathcal{R}$, la soluzione a costo minimo (ovvero quella ottenibile tramite l'azione $\mathcal{L}$) verrebbe ignorata e mai individuata. La soluzione che si può adottare per evitare ridondanza corrisponde al tenere traccia di tutti gli stati già visitati, come accade nel seguente esempio.

Graph-Search Algorithm

```

function GRAPH-SEARCH(problema, frontiera) returns una soluzione, o failure
    esplorati ← INSIEME-VUOTO()
    frontiera ← INSERISCI(CREA-NODO(problema.stato_iniziale))
    while not VUOTA(frontiera) do
        nodo ← POP(frontiera)
        if problema.GOAL_TEST(nodo.stato) then return nodo
        if nodo.stato not in esplorati then
            AGGIUNGI(nodo.stato, esplorati)
            frontiera ← INSERISCI_TUTTI(ESPANDI(nodo, problema))
        end if
    end loop
    return failure

```

Notiamo che se abbiamo un nodo con uno stato già esplorato, allora in questo caso non si procede con l'espansione. Nonostante ciò vi è il rischio di perdere la soluzione ottimale. Quindi il controllo del goal test e la natura di gestione della frontiera risultano essere fondamentali.

3.4 Ricerca Non Informata

3.4.1 Strategie Possibili

Una **Ricerca Non Informata** utilizza solo le informazioni disponibili nella definizione del problema.

Di seguito un elenco delle strategie possibili:

- Breadth-First Search
- Depth-First Search
- Depth-Limited Search
- Uniform-Cost Search (Dijkstra)
- Iterative Deepening Search

3.4.2 Breadth-First Search

La **Breadth-First Search** (BFS, ricerca in ampiezza) espande sempre il nodo **più superficiale** tra quelli presenti nella frontiera: prima la radice, poi tutti i suoi figli, poi tutti i figli dei figli e così via. L'albero viene quindi esplorato *livello per livello*: nessun nodo a profondità $k + 1$ viene espanso prima che siano stati espansi tutti i nodi a profondità k .

Per ottenere questo comportamento basta usare come frontiera una **coda FIFO**: i successori appena generati vengono inseriti in fondo alla coda,

mentre il nodo da espandere viene estratto dalla testa. Poiché i nodi di un livello vengono tutti generati prima di quelli del livello successivo, la coda risulta automaticamente ordinata per profondità. L'ordine con cui vengono generati i fratelli (cioè i nodi dello stesso livello) non è invece rilevante per il funzionamento dell'algoritmo.

Breadth-First Graph Search

```
function BFS(problema) returns una soluzione, o failure
  nodo ← CREA-NODO()
  nodo.stato ← problema.stato_iniziale
  nodo.path_cost ← 0
  if problema.GOAL_TEST(nodo.stato) then return nodo
  frontiera ← CREA-FIFO()
  frontiera ← INSERISCI(nodo)
  esplorati ← INSIEME-VUOTO()
  loop do
    if VUOTA(frontiera) then return failure
    nodo ← POP(frontiera) // il nodo più superficiale
    AGGIUNGI(nodo.stato, esplorati)
    for each azione in problema.AZIONI(nodo.stato) do
      figlio ← NODO-FIGLIO(problema, nodo, azione)
      if figlio.stato not in esplorati and figlio.stato not in frontiera then
        if problema.GOAL_TEST(figlio.stato) then return figlio
        frontiera ← INSERISCI(figlio)
      end if
    end for
  end loop
```

La funzione NODO-FIGLIO corrisponde a una singola iterazione del ciclo di ESPANDI: crea il nodo ottenuto applicando azione allo stato di nodo, impostandone padre, azione, stato, path cost e profondità.

Rispetto allo schema generale di GRAPH-SEARCH si notano tre differenze:

1. **La frontiera è una coda FIFO:** è l'unico elemento che caratterizza davvero la strategia.
2. **Il goal test viene anticipato:** non si controlla il nodo quando viene estratto dalla frontiera, ma *nel momento in cui viene generato* (per la radice, prima ancora di entrare nel ciclo).
3. **Il controllo sugli stati ripetuti riguarda anche la frontiera:** un figlio viene scartato non solo se il suo stato è già stato espanso, ma anche se è già presente in un nodo in attesa nella frontiera. In questo modo ogni stato compare al più in un nodo, e non si espandono mai due nodi con lo stesso stato: si tratta a tutti gli effetti di una *Graph Search*.

Approfondimento – Early Goal Test

Il controllo effettuato subito dopo la generazione prende il nome di **early goal test**. Nella BFS è lecito perché i nodi vengono generati in ordine di profondità: il primo nodo obiettivo generato è necessariamente quello alla profondità minima, quindi aspettare di estrarlo dalla frontiera non potrebbe portare a una soluzione più corta. Si risparmia però la generazione di un intero livello dell'albero (si veda la complessità temporale).

Come visto nella sezione sugli stati ripetuti, lo stesso ragionamento **non** vale se l'obiettivo è minimizzare il *path cost* con *step cost* diversi: in quel caso il primo nodo obiettivo generato non è necessariamente il più economico, e il goal test va effettuato all'estrazione (si veda la Uniform-Cost Search).

Approfondimento – Controllo degli stati nella frontiera

La frontiera contiene *nodi*, non stati: la condizione `figlio.stato not in frontiera` è un'abbreviazione tipica dello pseudocodice e significa “non esiste nella frontiera un nodo il cui attributo `stato` coincide con `figlio.stato`”. Scorrere la coda a ogni controllo costerebbe tempo lineare; nelle implementazioni reali si affianca quindi alla coda un insieme (ad esempio una hash table) contenente gli stati dei nodi in frontiera, così che il controllo avvenga in tempo costante.

Proprietà

- **Completezza:** sì, se il fattore di diramazione b è finito. Se una soluzione esiste, si trova a una profondità finita d , e la BFS raggiunge ogni livello dopo aver esaurito i precedenti, che sono composti da un numero finito di nodi.
- **Complessità temporale:** nel caso pessimo il nodo obiettivo è l'ultimo a essere generato al livello d . Grazie all'early goal test l'algoritmo si ferma proprio in quel momento, dopo aver generato

$$N = 1 + b + b^2 + \dots + b^d = \frac{b^{d+1} - 1}{b - 1}$$

nodi. Nella notazione asintotica conta solo il termine di grado massimo, quindi la complessità temporale è $O(b^d)$.

Se invece il goal test venisse effettuato all'estrazione (dopo la POP), prima di arrivare al nodo obiettivo bisognerebbe espandere tutti gli altri nodi del livello d , generando fino a $b \cdot (b^d - 1) = b^{d+1} - b$ nodi

del livello $d + 1$. La complessità temporale salirebbe così a $O(b^{d+1})$: un intero livello in più, cioè un fattore b .

- **Complessità spaziale:** ogni nodo generato rimane in memoria, o nell'insieme esplorati ($O(b^{d-1})$ stati) o nella frontiera ($O(b^d)$ nodi). Lo spazio è quindi dominato dalla frontiera ed è $O(b^d)$, come il tempo. È il vero punto debole della BFS: per problemi anche di dimensione moderata la memoria si esaurisce molto prima del tempo a disposizione.
- **Ottimalità:** la BFS trova sempre la soluzione con il *minor numero di passi*, ma questa coincide con quella a costo minimo solo se tutti gli step cost sono uguali (più in generale, se il path cost è una funzione non decrescente della profondità). Con step cost arbitrari non è ottima.

3.4.3 Uniform-Cost Search

La BFS è ottima solo quando tutti gli step cost sono uguali, perché espande i nodi in ordine di *profondità*. La **Uniform-Cost Search** (UCS, ricerca a costo uniforme) generalizza l'idea: espande sempre il nodo non ancora espanso con il **path cost** $g(n)$ **minimo**, cioè il nodo più “vicino” alla radice in termini di costo invece che di numero di passi. È la versione per la ricerca dell'algoritmo di **Dijkstra** per i cammini minimi.

L'implementazione si ottiene sostituendo la coda FIFO con una **coda con priorità** ordinata per path cost crescente: la POP restituisce il nodo con $g(n)$ minore. Se tutti gli step cost sono uguali, ordinare per costo equivale a ordinare per profondità, e la UCS si comporta esattamente come una BFS.

Uniform-Cost Search

```
function UNIFORM-COST-SEARCH(problema) returns una soluzione, o failure
  nodo ← CREA-NODO()
  nodo.stato ← problema.stato_iniziale
  nodo.path_cost ← 0
  frontiera ← CREA-CODA-PRIORITA() // ordinata per path_cost crescente
  frontiera ← INSERISCI(nodo)
  esplorati ← INSIEME-VUOTO()
  loop do
    if VUOTA(frontiera) then return failure
    nodo ← POP(frontiera) // il nodo con path_cost minimo
    if problema.GOAL_TEST(nodo.stato) then return nodo
    AGGIUNGI(nodo.stato, esplorati)
    for each azione in problema.AZIONI(nodo.stato) do
      figlio ← NODO-FIGLIO(problema, nodo, azione)
      if figlio.stato not in esplorati and figlio.stato not in frontiera then
        frontiera ← INSERISCI(figlio)
      else if figlio.stato in frontiera con path_cost maggiore then
        sostituisci quel nodo della frontiera con figlio
      end if
    end for
  end loop
```

Rispetto alla BFS vista prima, le modifiche necessarie per garantire l'ottimalità sono due:

1. **Il goal test si effettua quando il nodo viene selezionato per l'espansione** (subito dopo la POP), non quando viene generato.
2. **Se lo stato di un figlio è già presente nella frontiera con un path cost maggiore, il vecchio nodo viene sostituito** da quello nuovo, che rappresenta un cammino più economico verso lo stesso stato.

Approfondimento – Perché niente early goal test

Nella BFS l'early goal test funziona perché i nodi vengono *generati* in ordine di profondità: il primo nodo obiettivo generato è anche il meno profondo. Nella UCS, invece, l'ordine che conta è quello del path cost, e i nodi vengono generati in un ordine che non ha nulla a che vedere con il loro costo: quando viene generato un nodo obiettivo, nella frontiera possono esserci ancora nodi con g minore da cui parte un cammino più economico verso lo stesso obiettivo. Fermarsi alla generazione significherebbe restituire il *primo* cammino trovato, non il *migliore*.

Con il goal test all'estrazione, invece, un nodo obiettivo con costo g viene restituito solo quando è il nodo più economico dell'intera frontiera. Poiché gli step cost sono positivi, il costo lungo un cammino può solo crescere: ogni altro cammino non ancora completato costa già almeno g , e non può portare a una soluzione migliore. È esattamente il ragionamento visto nella sezione sugli stati ripetuti, con la figura delle azioni $\mathcal{R}$ e $\mathcal{L}$.

Proprietà

Sia C^* il costo della soluzione ottima e $\epsilon > 0$ il minimo step cost.

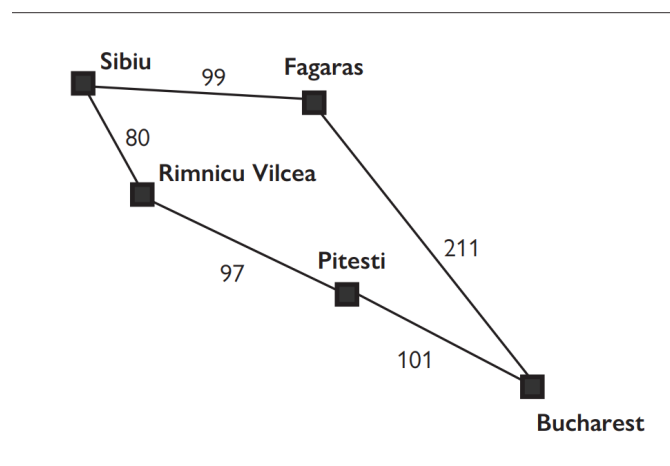
- **Completezza:** sì, se ogni step cost è $\geq \epsilon$ per qualche $\epsilon > 0$. Con step cost nulli la ricerca potrebbe bloccarsi in un ciclo infinito di azioni a costo zero (ad esempio un'azione *NoOp* che lascia lo stato invariato), perché il path cost non crescerebbe mai.
- **Complessità temporale:** la UCS espande tutti i nodi con $g \leq C^*$. Poiché ogni passo costa almeno ϵ , un cammino di costo C^* ha al più $\lfloor C^*/\epsilon \rfloor$ passi: questo valore prende il posto della profondità d della BFS. Il +1 all'esponente deriva dal goal test all'estrazione, che costringe a generare anche i figli dei nodi dell'ultimo livello. La complessità temporale è quindi

$$O(b^{1+\lfloor C^*/\epsilon \rfloor})$$

Se tutti gli step cost valgono ϵ , si ha $C^*/\epsilon = d$ e si ottiene $O(b^{d+1})$, cioè lo stesso costo della BFS con goal test all'estrazione. Se invece esistono step cost molto piccoli, C^*/ϵ può essere molto più grande di d : la UCS può esplorare a lungo cammini fatti di tanti passi economici prima di considerare passi più costosi ma utili.

- **Complessità spaziale:** come per la BFS, i nodi generati restano in memoria, quindi anche lo spazio è $O(b^{1+\lfloor C^*/\epsilon \rfloor})$.
- **Ottimalità:** sì. I nodi vengono espansi in ordine crescente di $g(n)$, quindi il primo nodo obiettivo estratto dalla frontiera è quello con il cammino a costo minimo.

Esempio: per capire perché sono necessari questi due accorgimenti, si consideri la seguente porzione del grafo della Romania.



Si vuole arrivare a Bucarest partendo da Sibiu, lungo il percorso a costo minimo. Accanto a ogni nodo in frontiera è indicato tra parentesi il suo path cost g .

1. **Inizializzazione.** La frontiera contiene solo il nodo con lo stato iniziale:
 - frontiera = {Sibiu (0)}
 - esplorati = $\emptyset$
2. **Si estrae Sibiu (0).** Non è un goal state, quindi viene aggiunto agli esplorati e si espande. Si generano i nodi per i suoi successori, iniziandone stato, padre, azione e path cost (essendo figli della radice, il path cost coincide con lo step cost):
 - Rimnicu Vilcea: step cost 80, path cost $0 + 80 = 80$

- Fagaras: step cost 99, path cost $0 + 99 = 99$

Stato dopo l'iterazione:

- frontiera = {Rimnicu Vilcea (80), Fagaras (99)}
- esplorati = {Sibiu}

3. **Si estrae Rimnicu Vilcea (80).** Non è un goal state: viene aggiunto agli esplorati e si espande.

- Pitesti: step cost 97, path cost $80 + 97 = 177$
- Sibiu: step cost 80, path cost $80 + 80 = 160$, ma è già esplorato, quindi il nodo viene scartato

Stato dopo l'iterazione:

- frontiera = {Fagaras (99), Pitesti (177)}
- esplorati = {Sibiu, Rimnicu Vilcea}

4. **Si estrae Fagaras (99).** Non è un goal state: viene aggiunto agli esplorati e si espande.

- Sibiu: step cost 99, path cost $99 + 99 = 198$, già esplorato, quindi scartato
- Bucarest: step cost 211, path cost $99 + 211 = 310$

Stato dopo l'iterazione:

- frontiera = {Pitesti (177), Bucarest (310)}
- esplorati = {Sibiu, Rimnicu Vilcea, Fagaras}

Primo accorgimento. Bucarest è appena stato generato, ma il goal test non viene fatto ora. Con l'*early goal test* la ricerca si fermerebbe qui restituendo il cammino da 310, senza considerare soluzioni meno costose e perdendo l'ottimalità.

5. **Si estrae Pitesti (177).** Non è un goal state: viene aggiunto agli esplorati e si espande.

- Bucarest: step cost 101, path cost $177 + 101 = 278$
- Rimnicu Vilcea: step cost 97, path cost $177 + 97 = 274$, già esplorato, quindi scartato

Secondo accorgimento. Bucarest è già presente in frontiera, ma con un path cost più alto ($310 > 278$): il vecchio nodo viene **sostituito** da quello nuovo. Stato dopo l'iterazione:

- frontiera = {Bucarest (278)}

- esplorati = {Sibiu, Rimnicu Vilcea, Fagaras, Pitesti}

6. **Si estrae Bucarest (278).** Il suo stato è il goal state: la ricerca termina e restituisce questo nodo.

La soluzione si ricostruisce risalendo l'attributo padre a partire dal nodo obiettivo: Sibiu → Rimnicu Vilcea → Pitesti → Bucarest, con costo 278.

3.4.4 Depth-First Search

La DFS espande sempre il nodo **più profondo** della frontiera, che viene implementata con una politica **LIFO** (stack): i successori vengono inseriti in cima e il prossimo nodo da espandere è l'ultimo inserito.

Un aspetto importante è la **rimozione dei nodi**: quando il nodo raggiunto non contiene un goal state e non possiede successori, può essere rimosso dalla memoria. Allo stesso modo, anche un sottoalbero già visitato interamente in cui non è presente un goal state può essere rimosso. Pertanto

$$\# \text{ nodi in memoria} \leq \# \text{ nodi generati}$$

e la complessità spaziale risulta molto più bassa di quella temporale.

Proprietà

- **Completezza:** no. Fallisce se lo spazio degli stati ha profondità infinita (può scendere all'infinito lungo un ramo senza soluzioni) e in presenza di stati ripetuti (può rimanere bloccata in un ciclo).
- **Complessità temporale:** con fattore di diramazione b e profondità massima m , nel caso pessimo vengono generati

$$1 + b + b^2 + \dots + b^m$$

nodi, ossia $O(b^m)$. È potenzialmente terribile, soprattutto se $m \gg d$. Se però le soluzioni sono *dense* (molti nodi contengono un goal state), la DFS può trovarne una molto prima della BFS.

- **Complessità spaziale:** $O(b \cdot m)$, ossia **lineare** nella profondità. Ogni nodo espanso genera b figli, ma se ne segue uno solo, lasciando in sospeso gli altri $b - 1$. Questo accade per ciascuno dei livelli da 1 a $m - 1$, mentre all'ultimo livello si trovano i b figli dell'ultimo nodo espanso. La memoria occupata è quindi proporzionale a

$$(m - 1) \cdot (b - 1) + b$$

Inoltre, quando si termina l'esplorazione di un ramo senza aver trovato un goal state, il ramo viene rimosso e si torna al nodo padre per esplorarne un altro (*backtracking*): la memoria liberata viene riutilizzata.

- **Ottimalità:** no. Restituisce la prima soluzione trovata lungo il ramo che sta esplorando, che può essere più profonda e più costosa di un'altra presente in un ramo diverso.

3.4.5 Iterative Deepening Search

È un metodo ibrido tra BFS e DFS: si eseguono ripetutamente delle DFS con un **limite di profondità** crescente. Prima si effettua una DFS con limite pari a 0 (visitando solo il nodo radice), poi con limite 1, poi 2 e così via, fino ad arrivare alla profondità in cui si trova il primo nodo con un goal state.

La tecnica è ibrida perché a ogni iterazione si visitano tutti i nodi con profondità minore o uguale al limite fissato, ottenendo un comportamento molto simile a quello della BFS (i livelli vengono raggiunti in ordine). Ogni singola ricerca, però, è una DFS e ne mantiene le caratteristiche, in particolare la complessità spaziale lineare.

Di seguito lo pseudocodice, che però assume il comportamento di TREE-SEARCH, consentendo l'esplorazione di nodi con stati ripetuti.

Depth-Limited Search

```
function DLS(problema, limite) returns una soluzione, o failure, o cutoff
  return R-DLS(CREA-NODO(problema.stato_iniziale), problema, limite)

function R-DLS(nodo, problema, limite) returns una soluzione, o failure, o cutoff
  if problema.GOAL_TEST(nodo.stato) then return nodo
  else if limite = 0 then return cutoff           // caso base
  else                                           // caso ricorsivo
    cutoff_avvenuto ← false
    for each azione in problema.AZIONI(nodo.stato) do
      figlio ← NODO-FIGLIO(problema, nodo, azione)
      risultato ← R-DLS(figlio, problema, limite - 1)
      if risultato = cutoff then cutoff_avvenuto ← true
      else if risultato ≠ failure then return risultato
    end for
    if cutoff_avvenuto then return cutoff
    else return failure
  end if
```

Iterative Deepening Search

```
function IDS(problema) returns una soluzione, o failure
  for profondità ← 0 to ∞ do
    risultato ← DLS(problema, profondità)
    if risultato ≠ cutoff then return risultato
  end for
```

Proprietà

- **Completezza:** sì, se il fattore di diramazione b è finito. Poiché si effettuano DLS con limite di profondità via via crescente (fino a ∞), se esiste una soluzione a profondità d la si trova all'iterazione con limite d .
- **Complessità temporale:** a ogni nuova DLS i nodi dei livelli superiori vengono rigenerati da capo, e le iterazioni sono fino a $d + 1$ se la soluzione è presente (altrimenti fino a $m + 1$). Il costo è quindi maggiore rispetto alla BFS, ma comunque dello stesso ordine di grandezza:

$$N_{\text{IDS}} = \underbrace{(d + 1) \cdot 1 + d b^1 + (d - 1) b^2 + (d - 2) b^3 + \dots + 1 \cdot b^d}_{\text{radice}} = O(b^d)$$

I nodi a profondità i vengono generati $d + 1 - i$ volte, una per ogni iterazione con limite $\geq i$. In particolare il nodo con lo stato iniziale viene generato a ogni iterazione, cioè $d + 1$ volte: una volta in più dei nodi a profondità 1.

- **Complessità spaziale:** $O(b \cdot d)$, ossia lineare. Ogni iterazione è una DFS con profondità massima pari al limite, quindi vale lo stesso ragionamento della DFS con d al posto di m : l'ultima iterazione, quella con limite d , è la più costosa in memoria. Notiamo comunque che gli stessi nodi vengono ricreati più volte, ma poiché al termine di ogni esecuzione di DLS la memoria viene liberata, lo spazio occupato resta sempre lineare.
- **Ottimalità:** sì, se tutti gli step cost sono uguali (più in generale, se il path cost è una funzione non decrescente della profondità). Come la BFS, la IDS raggiunge i livelli in ordine e trova quindi per prima la soluzione meno profonda.